

ITSAR2 LAB 1: DATA INTEGRATION AND ETL

Group Members:	Curio, Josh Nathan
	Gilera, Rowena
	Gromea, Nehje John
	Guanzon, Jurriel
	Sildora, Jegrick
Year & Section:	BSIT – 3B

Task 1:

- 1. What fields are nested objects (not plain values)?
Nested fields: address and company (note: address.geo is nested inside address).
- 2. What is the value of data[0].address.city?
Value: data[0].address.city = Gwenborough

Task 2:

- 1. What does .split(' ')[0] do to the phone number?
.split(' ')[0]: Keeps the substring before the first space — removes extensions (e.g., x56442) or extra text after the main phone number.
- 2. Why do we use parseFloat() on lat and lng?
parseFloat(lat/lng): Converts latitude/longitude from strings to numeric values for arithmetic, sorting, or filtering.
- 3. The filter step removes records without '@' in the email. How many records passed the filter?
Records passed filter: 10

Task 3:

- 1. How many columns does the CSV have? List them.
Columns: 11 — id, name, username, email, phone, city, zipcode, lat, lng, company, postCount
- 2. What is the company name of the user with id = 5?
Company (id = 5): Keebler LLC
- 3. What does fs.writeFileSync() do? What would happen if you used fs.appendFileSync() instead?
fs.writeFileSync(): Synchronously writes the file and overwrites any existing file at that path. Using fs.appendFileSync() would append data to the file (so repeated runs would accumulate rows instead of replacing them).

Challenge A:

- **What I did:** Fetched /posts, counted posts per userId, added postCount to each user row.
- **Result on JSONPlaceholder:** Each user has postCount = 10.

Challenge B:

- **Behavior implemented:** main().catch(...) prints a friendly error message ([ERROR] ETL pipeline failed: <message>) and exits with non-zero status.

- **If API is down / invalid URL:** The `extract()` call throws (non-ok response or network error), no CSV is written, and the error message is logged.

Challenge C:

- **How to implement:** After building rows, add `const southern = rows.filter(r => r.lat < 0)`; and return southern (or apply the filter before the email filter as needed).
- **How many remain (JSONPlaceholder data):** 7 users (ids 1,2,3,5,6,8,10).

TERMINAL OUTPUT:

```
PS C:\Users\localhost\Documents\ITSAR2\etl-lab>node lab_node.js
[EXTRACT] Fetching: https://jsonplaceholder.typicode.com/users
[EXTRACT] Got 10 records.

[TASK 1] First user object (raw):
{
  id: 1,
  name: 'Leanne Graham',
  username: 'Bret',
  email: 'Sincere@april.biz',
  address: {
    street: 'Kulas Light',
    suite: 'Apt. 556',
    city: 'Gwenborough',
    zipcode: '92998-3874',
    geo: { lat: '-37.3159', lng: '81.1496' }
  },
  phone: '1-770-736-8031 x56442',
  website: 'hildegard.org',
  company: {
    name: 'Romaguera-Crona',
    catchPhrase: 'Multi-layered client-server neural-net',
    bs: 'harness real-time e-markets'
  }
}

[TASK 1] Nested Fields: "address" and "company"
[TASK 1] data[0].address.city = Gwenborough

[EXTRACT] Fetching: https://jsonplaceholder.typicode.com/posts
[EXTRACT] Got 100 records.
[TRANSFORM] Cleaning 10 records...
[TRANSFORM] Clean records: 10

[TRANSFORM] Preview of clean data:
```

(index)	id	name	username	email	phone	city	zipcode	lat	lng	company	postCount
0	1	'Leanne Graham'	'bret'	'sincere@april.biz'	'1-770-736-8031'	'Gwenborough'	'92998-3874'	-37.3159	81.1496	'Romaguera-Crona'	10
1	2	'Ervin Howell'	'antonette'	'shanna@melissa.tv'	'010-692-6593'	'Wisokyburgh'	'90566-7771'	-43.9509	-34.4618	'Deckow-Crist'	10
2	3	'Clementine Bauch'	'samantha'	'nathan@yesenia.net'	'1-463-123-4447'	'McKenziehaven'	'59590-4157'	-68.6102	-47.0653	'Romaguera-Jacobson'	10
3	4	'Patricia Lebsack'	'karianne'	'julianne.oconner@kory.org'	'493-170-9623'	'South Elvis'	'53919-4257'	29.4572	-164.299	'Robel-Corkery'	10
4	5	'Chelsey Dietrich'	'kamren'	'lucio.hettinger@annie.ca'	'(254)994-1289'	'Roscoeview'	'33263'	-31.8129	62.5342	'Keebler LLC'	10
5	6	'Mrs. Dennis Schulist'	'karley_dach@jasper.info'	'1-477-935-8478'	'71477-935-8478'	'South Christy'	'23880-1370'	-71.4197	71.7478	'Considine-Lockman'	10
6	7	'Glenn Weissnat'	'telly.hoeger@billy.biz'	'210-467-6132'	'210-467-6132'	'Houemouth'	'38884-1099'	24.8918	21.8904	'Johns Group'	10
7	8	'Nicholas Runolfsdottir V'	'maxime_nienow'	'sherwood@rosamond.me'	'586.493.6943'	'Aliyaview'	'45169'	-14.399	-128.7677	'Abernathy Group'	10
8	9	'Glenra Reichert'	'delphine'	'chaim.mcdermott@dana.io'	'(775)976-6794'	'Bartholomebury'	'76495-3109'	24.6463	-168.8889	'Vost and Sons'	10
9	10	'Clementina DuBuque'	'moriah.stanton'	'rey.padberg@karina.biz'	'024-648-3804'	'Lebsackbury'	'31428-2261'	-38.2386	57.2232	'Hoeger LLC'	10

```
[LOAD] Writing 10 rows to CSV...
[LOAD] Saved to output.csv
[LOAD] File_path: C:\Users\localhost\Documents\ITSAR2\etl-lab\output.csv
```

MS EXCEL OUTPUT (output.csv)

[illegible]

CODE:

```
1 // etl.js - Lab 1: Fetch & ETL with Node.js
2
3 const fs = require("fs");
4 const path = require("path");
5
6 const API_URL = "https://jsonplaceholder.typicode.com/users";
7 const POSTS_API_URL = "https://jsonplaceholder.typicode.com/posts";
8
9 // — 1. EXTRACT —————
10 async function extract(url) {
11   console.log("[EXTRACT] Fetching:", url);
12
13   const response = await fetch(url);
14
15   if (!response.ok) {
16     throw new Error("HTTP error: " + response.status);
17   }
18
19   const data = await response.json();
20   console.log("[EXTRACT] Got", data.length, "records.");
21
22   // Task 1: Inspect the first user object structure
23   if (url === API_URL) {
24     console.log("\n[TASK 1] First user object (raw):");
25     console.log(data[0]);
26     console.log('[TASK 1] Nested fields: "address" and "company"');
27     console.log("[TASK 1] data[0].address.city =", data[0].address.city);
28     console.log();
29   }
30
31   return data;
32 }
33
34 // — 2. TRANSFORM —————
35 function transform(users, posts) {
36   console.log("[TRANSFORM] Cleaning", users.length, "records ...");
37
38   // Challenge A: count posts per userId
39   const postCounts = {};
40   if (posts) {
41     for (const post of posts) {
42       postCounts[post.userId] = (postCounts[post.userId] || 0) + 1;
43     }
44   }
45
46   const rows = users.map((user) => {
47     const row = {
48       id: user.id,
49       name: user.name.trim(),
50       username: user.username.toLowerCase(),
51       email: user.email.toLowerCase(),
52       phone: user.phone.split(" ")[0], // keep only the main number
53       city: user.address.city,
54       zipcode: user.address.zipcode,
55       lat: parseFloat(user.address.geo.lat),
56       lng: parseFloat(user.address.geo.lng),
57       company: user.company.name,
58     };
59
60     // Challenge A: add postCount column if posts data is available
61     if (posts) {
62       row.postCount = postCounts[user.id] || 0;
63     }
64
65     return row;
66   });
67
68   // Filter out any records with invalid email
69   const clean = rows.filter((r) => r.email.includes("@"));
70
71   console.log("[TRANSFORM] Clean records:", clean.length);
72   return clean;
73 }
74
```

```

75 // — 3. LOAD —————
76 function load(rows) {
77   console.log("[LOAD] Writing", rows.length, "rows to CSV...");
78
79   // Build the header row from the keys of the first object
80   const headers = Object.keys(rows[0]);
81
82   // Build each data row — wrap values in quotes to handle commas inside values
83   const lines = rows.map((row) => {
84     return headers.map((h) => `"${row[h]}"`).join(",");
85   });
86
87   // Join header + data rows with newlines
88   const csv = [headers.join(","), ...lines].join("\n");
89
90   // Write to file
91   const filePath = path.join(__dirname, "output.csv");
92   fs.writeFileSync(filePath, csv, "utf8");
93
94   console.log("[LOAD] Saved to output.csv");
95   console.log("[LOAD] File path:", filePath);
96 }
97
98 // — MAIN —————
99 async function main() {
100   // Exercise 1 + Challenge A: fetch both users and posts
101   const raw = await extract(API_URL);
102   const posts = await extract(POSTS_API_URL);
103
104   // Exercise 2: transform with post counts
105   const clean = transform(raw, posts);
106
107   // Exercise 2: preview clean data in terminal
108   console.log("\n[TRANSFORM] Preview of clean data:");
109   console.table(clean);
110
111   // Exercise 3: write CSV
112   load(clean);
113 }
114
115 main().catch((err) => {
116   // Challenge B: friendly error handling
117   console.error("[ERROR] ETL pipeline failed:", err.message);
118   process.exit(1);
119 });
120

```